DV and Performance Tracks

Lab T-MOD: Transaction-Level Cache Model

Object-Oriented SystemVerilog and Reference Models

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 24, 2026

LAB OBJECTIVE Write a run-time configurable, class-based, transaction-level model of the data cache that predicts the result, the read data and every memory transfer of each access, verify the model on its own before it is trusted, and package it so that the DV and architecture-modeling tracks can both build on it.

Duration	1.5 weeks (approximately 25–30 hours)
Prerequisites	Common core completed, in particular Labs R4 (`dcache`) and V3 (classes, constrained random, coverage)
Designs	`cache_model_pkg` (new, `dv/model/cache_model/`); `dcache` from R4 is read as a specification only
Reused in	T-DV1 and T-DV2 (golden reference in the scoreboard), T-AM1 (design-space exploration), T-AM2 (timing extension)

Contents

1 Overview **3**

2 Object-Oriented SystemVerilog **4**

2.1 Handles, Objects and Copies . . . 4

2.2 Inheritance, Virtual Methods and Polymorphism . . . 5

2.3 Abstract Classes . . . 5

2.4 Parameterized Classes and Packages . . . 6

2.5 Downcasting with `$cast` . . . 7

3 The Cache Model **7**

3.1 Class Structure . . . 7

3.2 Transaction Level: What the Model Predicts . . . 8

3.3 Configuration and Statistics . . . 9

3.4 Required API . . . 9

3.5 Requirements . . . 10

3.6 Replacement Policies . . . 11

4 Verifying the Model **11**
4.1 Step 1: Hand-Calculated Traces . . . 11
4.2 Step 2: Transparency Self-Check . . . 12
4.3 Step 3: Clone Check . . . 13
4.4 Bug-Injection Exercise for the Model . . . 13

5 Requirements and Tasks **13**

6 Repository and Flow **14**

7 Deliverables **15**

8 Assessment **16**

9 Review Questions **16**

10 References **17**

1 Overview

In Lab V3 the testbench checked the cache with three separate pieces: a flat memory model for read data, a tag model for hit/miss, and a final memory comparison after flush. Each piece knew only a part of the cache's behavior. In this lab you replace them with one **reference model**: a class that behaves like the cache at the level of transactions. For every CPU request it predicts the result, the read data, and the exact list of memory transfers the cache must perform.

The model is not tied to the RTL. It runs on its own, it is configured at run time, and it can be used for something the RTL cannot do quickly: exploring many cache configurations before any RTL exists. This is how processor teams use models in practice — first for architecture decisions by the architecture-modeling team, then as the golden reference for RTL verification by the DV team. For that reason this lab is the shared first lab of two tracks: the DV track (T-DV1, T-DV2) plugs the model into a scoreboard, and the architecture-modeling track (T-AM1, T-AM2) drives it with traces and extends it with timing.

Lab V3 used classes only as containers of random fields. Writing a model that two teams will reuse for months needs the rest of object-oriented SystemVerilog: handles and deep copies, inheritance and virtual methods, abstract classes, parameterized classes, packages and `$cast`. The first part of this lab teaches these features, and every one of them is used in the model.

Figure 1 shows the two ways the same package is used.

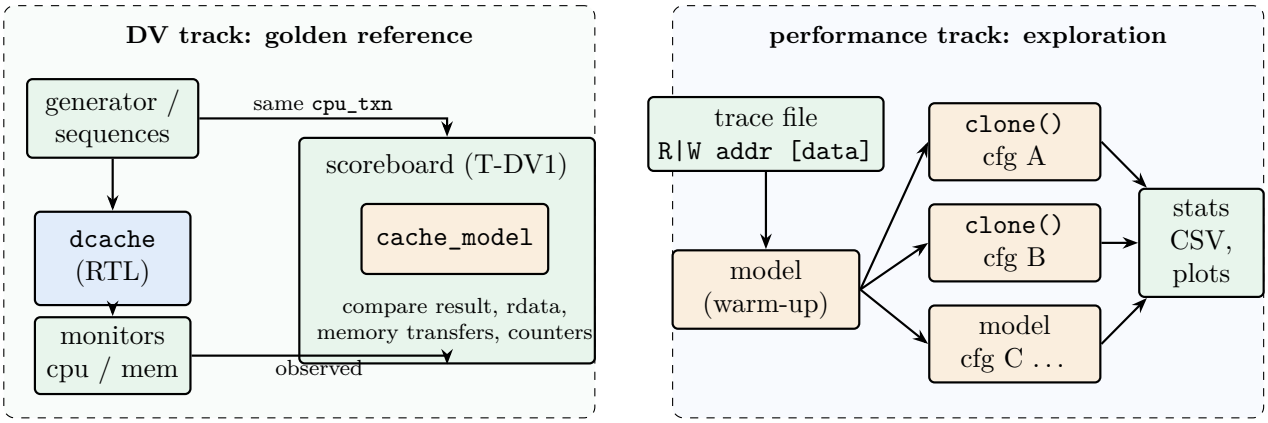


Figure 1: One package, two uses. Left: in the DV track the model receives the same CPU transactions as the RTL and the scoreboard compares its predictions with what the monitors observe. Right: in the performance track the model runs alone on traces, many configurations (or warm clones of one model) side by side, and only its statistics are kept.

Where this lab sits. It builds on the cache architecture of Lab R4 (address fields, write-back/write-allocate, performance counters) and on the class and randomization basics of Lab V3. It does *not* connect the model to the RTL — that is Lab T-DV1 — and it does not sweep configurations — that is Lab T-AM1. The deliverable of this lab is a verified package, `cache_model_pkg`, that both later labs import unchanged.

By the end of this lab you should be able to:

1. Distinguish handles from objects, and shallow copies from deep copies, and write a correct `clone()` for a class with nested objects and dynamic arrays.
2. Use inheritance, virtual methods, abstract classes, parameterized classes and static members, and explain polymorphism with an example from your own code.
3. Organize reusable classes in a package, and use `$cast` safely.
4. Write a transaction-level model and explain what it predicts and what it deliberately leaves out.

5. Implement replacement policies behind an abstract interface so that new policies need no change in the model.
6. Verify a reference model independently of the design it will check: hand traces, a transparency self-check and a clone check.

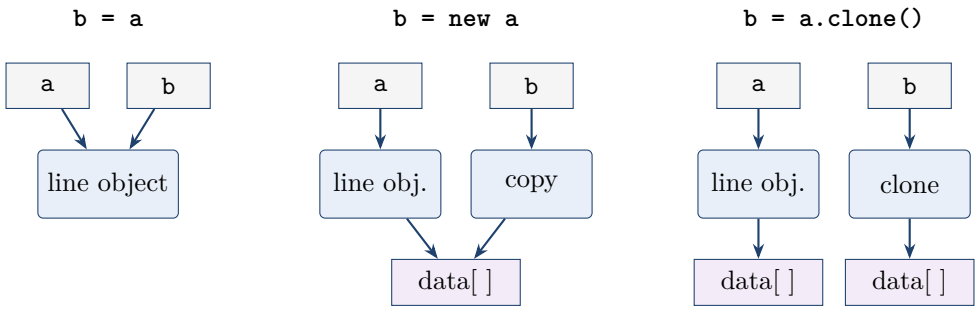
2 Object-Oriented SystemVerilog

This section covers the language features the model needs. Each subsection ends with the place in the model where the feature is used.

2.1 Handles, Objects and Copies

A class variable holds a *handle* — a reference to an object — not the object itself.

```
cpu_txn a, b;
a = new();          // one object
b = a;              // two handles, still ONE object: changing b changes a
b = new a;          // shallow copy: a new object, but object members inside
                    // are shared handles (only the top level is copied)
b = a.clone();      // deep copy: your own method, copies nested objects too
```



Note that a SystemVerilog dynamic array *member* is copied by value by `new a`; the trap is a member that is a *class handle* (a `cache_line`, a policy object, a memory model). The model stores cache lines as objects. A shallow copy of the model would share those line objects, and a later write to the "copy" would silently change the original's state. Every class in this lab that holds nested objects needs a deep `copy()`/`clone()` pair:

```
class cache_line #(int DW = 32);
  bit            valid, dirty;
  int unsigned   tag;
  logic [DW-1:0] data [];

  // copy(): fill THIS object from rhs (used by clone() and by derived classes)
  virtual function void copy(cache_line #(DW) rhs);
    valid = rhs.valid; dirty = rhs.dirty; tag = rhs.tag;
    data  = new[rhs.data.size()](rhs.data);    // new array, same contents
  endfunction

  virtual function cache_line #(DW) clone();
    clone = new();
    clone.copy(this);
  endfunction
endclass
```

Used in the model: `cache_line`, `cache_stats`, `mem_model`, every replacement policy and `cache_model` itself have `clone()`; the clone check of Section 4 proves they are deep.

2.2 Inheritance, Virtual Methods and Polymorphism

```
class cpu_txn #(int ADDR_W = 10, int DW = 32);
  rand bit                 we;
  rand logic [ADDR_W-1:0] addr;
  rand logic [DW-1:0]      wdata;
  static int unsigned      next_id = 0;   // shared by all objects
  int unsigned             id;

  function new();
    id = next_id++;
  endfunction

  virtual function string convert2string();
    return $sformatf("#%0d %s addr=%h", id, we ? "WR" : "RD", addr);
  endfunction

  virtual function cpu_txn #(ADDR_W, DW) clone();
    cpu_txn #(ADDR_W, DW) c = new();
    c.we = we; c.addr = addr; c.wdata = wdata;
    return c;
  endfunction
endclass
```

A derived class (`extends`) inherits all members and may override `virtual` methods; inside it, `super.f()` calls the base version and `this` refers to the current object. A handle of the base type may point to an object of a derived type; calling a virtual method through that handle runs the derived version. This is **polymorphism**. It is what lets the model call `policy.victim()` without knowing which policy it holds, and what lets a generator in Lab T-DV1 produce conflict or sequential transactions without knowing which kind it holds. Without `virtual`, the base version runs — a common and silent bug.

```
class tagged_txn #(int ADDR_W = 10, int DW = 32) extends cpu_txn #(ADDR_W, DW);
  string origin;                          // e.g. trace file and line number
  virtual function string convert2string();
    return {super.convert2string(), " from ", origin};
  endfunction
  // clone() MUST be overridden too, otherwise a clone is a plain cpu_txn
endclass
```

Static members (`next_id` above) exist once per class specialization, not once per object; use them for IDs and global counters, never for state that belongs to one model instance (two model instances in one simulation, as in Lab T-AM1, would share it).

Visibility. Mark internal state `protected` (visible in derived classes) or `local` (this class only). The model's storage is `protected`: the testbench may read it only through methods, which keeps T-DV1 and T-AM1 code independent of the model's internals.

2.3 Abstract Classes

An abstract (`virtual`) class cannot be instantiated; it defines an interface that derived classes must implement with `pure virtual` methods:

```
virtual class replacement_policy;
  pure virtual function void         reset(int unsigned sets, int unsigned ways);
  pure virtual function int unsigned victim(int unsigned set);
  pure virtual function void         touch(int unsigned set, int unsigned way);
  pure virtual function void         fill (int unsigned set, int unsigned way);
  pure virtual function string       name();
  pure virtual function replacement_policy clone();
endclass

class lru_policy extends replacement_policy;
  protected int unsigned age [][];     // [set][way], dynamic 2-D array
  // implement all six methods
endclass
```

The model holds a `replacement_policy` handle and never asks which policy it is. Adding a new policy means adding a class — the model does not change. Note the polymorphic `clone()`: the model clones its policy through the base handle and gets back an object of the right derived type.

2.4 Parameterized Classes and Packages

The model and its helper classes live in one package, `cache_model_pkg`, in `dv/model/cache_model/`, so that the DV track and the architecture-modeling track import the same code. Bit widths are class parameters (they must match the RTL ports at compile time); cache geometry is a run-time configuration object (it changes between runs, or even within one run in T-AM1).

```
// dv/model/cache_model/cache_model_pkg.sv
package cache_model_pkg;
  typedef enum {HIT, MISS_CLEAN, MISS_DIRTY} result_e;
  `include "cpu_txn.svh"
  `include "mem_txn.svh"
  `include "cache_line.svh"
  `include "cache_stats.svh"
  `include "cache_pred.svh"
  `include "mem_model.svh"
  `include "replacement_policy.svh"   // abstract base class
  `include "lru_policy.svh"           // one derived policy per file
  `include "fifo_policy.svh"
  `include "random_policy.svh"
  `include "cache_cfg.svh"            // after the policies: make_policy() creates them
  `include "cache_model.svh"
endpackage
```

Each specialization of a parameterized class is a distinct type: `cpu_txn#(10,32)` and `cpu_txn#(12,32)` cannot be assigned to each other. Use `typedef`s to keep names short and consistent:

```
import cache_model_pkg::*;
typedef cpu_txn     #(ADDR_W, DW) cpu_txn_t;
typedef cache_model #(ADDR_W, DW) cache_model_t;
```

A class must be declared before code that creates or uses it (or be announced with `typedef class name;`); this is why `cache_cfg`, whose `make_policy()` calls `new` of every policy, comes after them.

Rules for the package: no `module`, no virtual interface, no hierarchical reference, no `#` delay or `@` event control (the model is timeless), no `$urandom` in model code (see the random policy below). File names equal class names; class files have the extension `.svh` and are only included by the package.

2.5 Downcasting with `$cast`

Assigning a base handle to a derived handle needs a run-time check:

```
replacement_policy p = cfg.make_policy();   // returns a base handle
lru_policy         lru;
if (!$cast(lru, p))
  $fatal(1, "policy %s is not LRU", p.name());
```

`$cast` as a function returns 0 on failure and leaves the target unchanged; as a task it is a run-time error. Always use the function form and check it. Needing many downcasts is a design smell: if code must know the derived type, the method it wants probably belongs in the base class as a `virtual` method. `$cast` is also the legal way to convert an `int` into an enum (`$cast(res, 2)`).

Tool note: the package relies on Verilator's support for classes, parameterized classes, dynamic and associative arrays of class handles and `$cast`. The coordinator confirms the installed version before the lab starts. No constrained randomization is needed in the model itself.

SYNOPSYS CLOUD

Use VCS: `make -C dv/model/cache_model sim SIM=vcs` (the cloud default). VCS supports the complete class language of IEEE 1800 — parameterized classes, abstract classes, 2-D dynamic arrays of class handles, queues of handles, `$cast`, static members — and runs the model at about $3 \cdot 10^5$ accesses per second. The installed Verilator 5.013 does *not* compile the required `cache_line lines [][]` ("dynamic new() not expected in this context", then an internal error), nor queue methods on an array element (`order[set].push_back(w)`); keep policy state in flat arrays `[set * ways + way]` if you want that part portable. Lint: SpyGlass and VC SpyGlass lint synthesizable RTL only and cannot read a class package (SpyGlass reports syntax fatals; VC SpyGlass black-boxes the top and reports nothing — a false pass); lint with the VCS compiler instead, `make -C dv/model/cache_model lint LINT_TOOL=vcs` (`vcs +lint=all`, the block Makefile selects it).

3 The Cache Model

3.1 Class Structure

Figure 2 is the class diagram of the package: which classes inherit from which (open triangles), and which object creates and holds which (filled diamonds at the owner). Every class in it is introduced in Section 2.

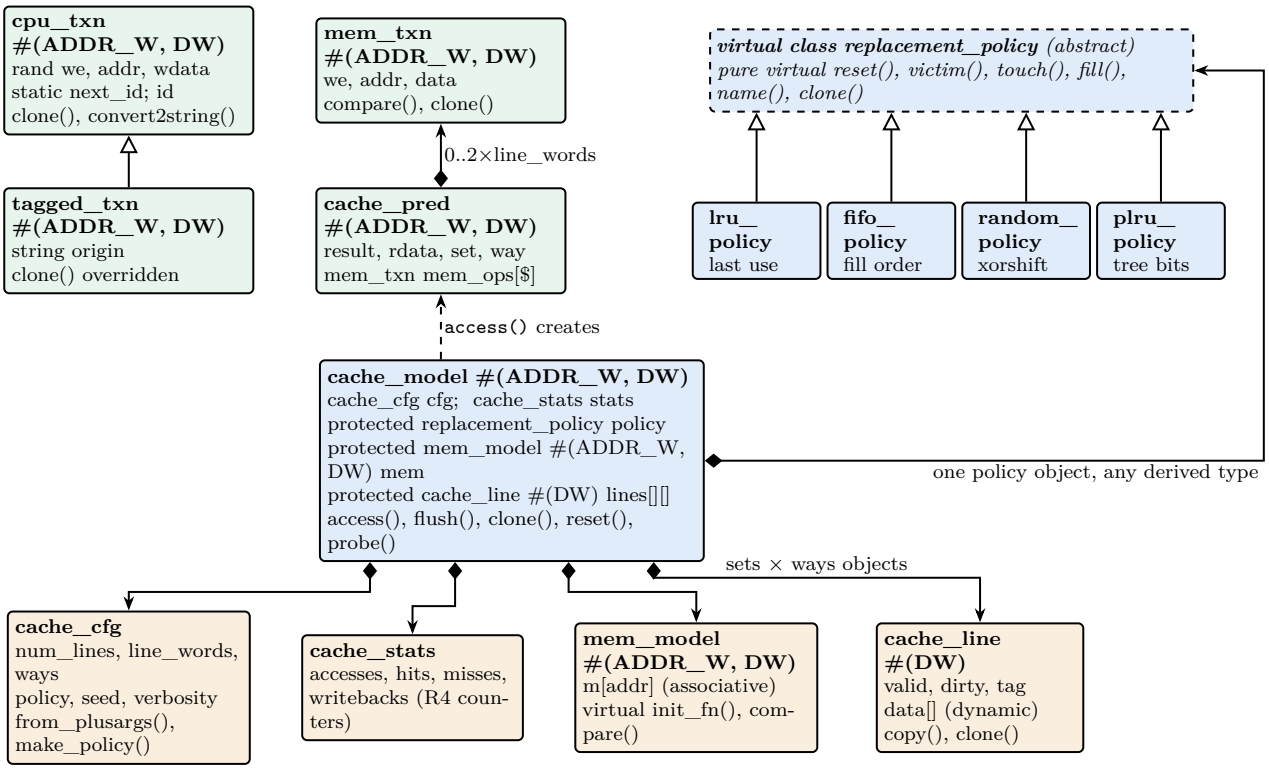


Figure 2: Class diagram of `cache_model_pkg`. Inheritance: `tagged_txn` from `cpu_txn`; every policy from the abstract `replacement_policy` (dashed box, italic: cannot be instantiated). Ownership: the model holds its configuration, statistics, policy, backing memory and `sets`×`ways` line objects, and `clone()` copies all of them deeply; each `access()` creates a new `cache_pred` holding the memory transactions. The model only knows `replacement_policy`; `cache_cfg::make_policy()` is the one place that names the derived classes. Parameterized classes carry `#(ADDR_W, DW)`.

3.2 Transaction Level: What the Model Predicts

The model has no clock. It processes one CPU transaction at a time and returns everything that transaction causes:

- the result: `HIT`, `MISS_CLEAN` or `MISS_DIRTY`;
- for a read, the data;
- the ordered list of memory transactions: the write-back of the victim line (if dirty) followed by the refill reads, with addresses and write data;
- updated statistics (accesses, hits, misses, write-backs), defined exactly like the R4 performance counters.

It deliberately does **not** predict *when* anything happens: not the number of stall cycles, not the cycle of a response. In the DV track, timing is checked by the interface assertions and the RTL design assertions; in the performance track, Lab T-AM2 adds timing *around* this model without changing it. Keeping timing out of the model keeps it simple and independent of micro-architectural choices — the same model stays valid if the RTL is pipelined differently.

Aspect	Predicted by the model	Not predicted (and who covers it)
Function	Read data; final memory contents after flush	—
Cache state	Valid, dirty, tag and data of every line; replacement order	Physical storage layout (RTL detail)
Memory traffic	Order, address, direction and write data of every transfer	Cycle of each transfer; back-pressure (assertions, T-AM2)
Performance	Hits, misses, write-backs (exact)	Stall cycles, latency, AMAT in cycles (T-AM2)
Flush	The *set* of write-backs	Order of lines during flush (not specified in R4)

3.3 Configuration and Statistics

```
class cache_cfg;
  int unsigned num_lines  = 16;    // total lines (power of two)
  int unsigned line_words = 4;     // words per line (power of two, >= 2)
  int unsigned ways       = 1;     // 1 = direct-mapped (the R4 RTL)
  string       policy     = "lru"; // "lru", "fifo", "random"
  int unsigned seed       = 1;     // for the random policy
  int unsigned verbosity  = 0;

  function void from_plusargs();   // +NUM_LINES= +LINE_WORDS= +WAYS= +POLICY= +
    CACHE_SEED=
  function bit  is_valid();        // powers of two, ways <= num_lines, ...
  function replacement_policy make_policy();   // the one place that knows names
  function string convert2string();
endclass

class cache_stats;
  longint unsigned accesses, hits, misses, writebacks; // writebacks in LINES
  function real hit_rate();  return accesses ? real'(hits) / accesses : 0.0;
  endfunction
endclass
```

Statistics follow the R4 counter definitions exactly, so that T-DV1 and T-AM1 can compare them with the RTL without translation: `accesses` counts CPU requests (each request once, even if it first missed); `misses` counts refills caused by CPU requests; `writebacks` counts dirty *lines* written to memory, including those written during a flush. `hits = accesses - misses`.

3.4 Required API

```
class cache_model #(int ADDR_W = 10, int DW = 32);
  cache_cfg               cfg;
  cache_stats             stats;
  protected replacement_policy policy;
  protected mem_model #(ADDR_W, DW) mem;      // backing memory contents
  protected cache_line #(DW) lines [][];      // [set][way]

  function new(cache_cfg cfg, mem_model #(ADDR_W, DW) mem);

  // Process one CPU transaction; update the state; return the prediction
  function cache_pred #(ADDR_W, DW) access(cpu_txn #(ADDR_W, DW) t);

  // Write back all dirty lines, invalidate all lines; return write-backs
```

```
  function void flush(ref mem_txn #(ADDR_W, DW) wbs [$]);

  // Deep copy of the complete model state (lines, policy, memory, statistics)
  function cache_model #(ADDR_W, DW) clone();

  function void   reset();            // size storage from cfg, invalidate, clear stats
  function string convert2string();   // dump of all valid lines
  // read-only queries for testbenches and exploration
  function mem_model #(ADDR_W, DW) get_mem();
  function bit    probe(logic [ADDR_W-1:0] addr);   // would it hit? no side effects
endclass

class cache_pred #(int ADDR_W = 10, int DW = 32);
  result_e                result;
  logic [DW-1:0]          rdata;      // valid for reads
  int unsigned            set, way;   // where the line now lives
  mem_txn #(ADDR_W, DW)   mem_ops[$]; // write-backs first, then refill reads
endclass
```

The access algorithm, for an address split into tag, set and offset with the R4 field definitions (`OFF_W = $clog2(line_words)`, `SET_W = $clog2(num_lines / ways)`):

1. Search the ways of the set for a valid line with a matching tag. If found: `HIT`, call `policy.touch(set, way)`.
2. Otherwise choose a way: an invalid way if one exists (lowest index first), else `policy.victim(set)`. If the victim is valid and dirty, append `line_words` write transactions (ascending word order, the victim's own tag) and count one write-back: `MISS_DIRTY`; else `MISS_CLEAN`.
3. Append `line_words` refill reads in ascending order, load the data from `mem`, set valid, clear dirty, store the tag, call `policy.fill(set, way)`.
4. Perform the request on the line: a read returns the word; a write updates it and sets dirty.

3.5 Requirements

M1 Geometry comes only from `cfg` at run time: `num_lines`, `line_words`, `ways`. The number of sets is `num_lines / ways`. Storage is dynamic arrays sized in `reset()`: `cache_line lines [][]` indexed by `[set][way]`. Changing `cfg` and calling `reset()` must give a fully working model of the new geometry. The model latches the geometry in `reset()`: `cfg` is a shared handle, and a `flush()` after `cfg.line_words` changed but before `reset()` must still use the old line size.

M2 `cache_line` is a class holding `valid`, `dirty`, `tag` and a dynamic array of data words, with a deep `copy()` and `clone()`.

M3 Replacement policies: `lru_policy`, `fifo_policy` and `random_policy`, all derived from the abstract `replacement_policy`. With `ways = 1`, every policy must choose way 0.

M4 `random_policy` must be reproducible from `cfg.seed` and independent of the simulator's thread random state: implement it with its own generator (for example a 32-bit xorshift whose state is a class member). Its `clone()` copies the generator state.

M5 Refill and write-back words are listed in ascending word order, matching the R4 RTL. If your R4 RTL uses a different order, record it in the README and agree with the coordinator which one changes; the model and the RTL must agree before T-DV1.

M6 `mem_model` is an associative array (`logic [DW-1:0] m [logic [ADDR_W-1:0]]`) with a configurable default for unwritten words (`init_fn(addr)`, by default the V3 preload pattern address XOR constant), and a `compare(other)` method.

M7 The model never reads the DUT, never uses a virtual interface, and contains no delays or event controls. It must compile and run in a testbench with no RTL at all.

M8 Every class has `convert2string()`; with `cfg.verbosity > 0`, `access()` prints its prediction in one line (used for debugging in T-DV1 and T-AM1).

3.6 Replacement Policies

With `ways = 1` (the R4 RTL) there is only one candidate and the policy is irrelevant; with `ways > 1` the policy decides the victim among the valid ways of a set.

Policy	State per set	`touch` (hit) / `fill` (refill)	`victim`
LRU	Age per way (or an ordered list)	Both make the way most recently used	Least recently used way
FIFO	Insertion order	`touch`: nothing; `fill`: newest	Oldest inserted way
Random	One generator for the whole cache	Nothing	Pseudo-random way from the generator

The distinction between `touch` and `fill` is the only reason the interface has both: for LRU they do the same thing, for FIFO only a fill changes the order. Pseudo-LRU (a tree of bits per set, as many real caches use for 4 or 8 ways) is a good extra policy; it must fit into the same interface without changing the model.

4 Verifying the Model

A wrong reference model is worse than none: it reports bugs that do not exist, or — far more dangerous — agrees with a wrong design. And a wrong model gives confident, wrong architecture decisions. Verify the model before it checks or predicts anything. All three steps run in `tb_model_unit`, a testbench with no RTL.

TEST TERM | Model correlation test

> The test this verification prepares: in T-AM1 the same trace runs on the model and on the R4 RTL and the counters must match exactly. A match only means something if the model was verified on its own first — otherwise model and RTL may share the same wrong idea of the cache.

4.1 Step 1: Hand-Calculated Traces

TEST TERM | Directed test

> Each hand trace is a directed test: a short, chosen sequence whose every result was computed on paper. It is the only step that checks the *policy* and the *traffic* (which line is evicted, which words move); a pass proves exactly the cases written down.

Feed short traces whose results you computed by hand, and check every predicted result, memory transaction and statistic. Example with a deliberately tiny configuration, `num_lines = 4`, `line_words = 2`, `ways = 1`, `ADDR_W = 6`: offset `a[0]`, index `a[2:1]`, tag `a[5:3]`.

#	Access	Set / tag	Predicted memory transactions	Result
1	`W 0x00 = AA`	0 / 0	`R 00`, `R 01`; line becomes dirty	`MISS_CLEAN`
2	`R 0x08`	0 / 1	`W 00=AA`, `W 01=mem[01]`, `R 08`, `R 09`	`MISS_DIRTY`
3	`R 0x01`	0 / 0	`R 00` (returns AA), `R 01`	`MISS_CLEAN`
4	`R 0x00`	0 / 0	none; data AA	`HIT`

Statistics: accesses 4, hits 1, misses 3, write-backs 1 (line).

A second example shows why the trace set must cover every policy with `ways = 2`: with `num_lines = 4`, `ways = 2` (two sets), lines `A = 0x00`, `B = 0x04`, `C = 0x08` all fall into set 0. The trace `A B A C A` gives:

Policy	A	B	A	C	A	Victim of C
LRU	miss	miss	hit	miss	hit	B (least recently used)
FIFO	miss	miss	hit	miss	miss	A (oldest inserted)

Write at least ten traces covering: clean and dirty misses, a write hit followed by an eviction, a miss on every word offset (the requested word is not always word 0), flush with zero, one and all lines dirty, and every replacement policy with `ways = 2`. For `random_policy`, check that the same seed gives the same victims and a different seed gives different victims. Store each trace and its expectation as data (a queue of structs or a small text file), not as hundreds of lines of copy-pasted checks.

4.2 Step 2: Transparency Self-Check

TEST TERM | Random test (self-checking)

> Long random traces checked against a trivially correct reference, a plain memory. The randomness reaches states no hand trace reaches (every offset, every eviction order, wrap-around of the address range); the seed makes a failure reproducible.

Software must not be able to tell whether a cache exists. Run long random traces (for example 10^5 accesses, address range two to eight times the capacity) through the model and, in parallel, through a plain associative-array memory updated by every write. Every read predicted by the model must equal the plain memory. After a final `flush()`, the model's backing memory must equal the plain memory word for word. This must hold for every configuration (at least the 48 combinations of `num_lines` $\in \{4, 16, 64, 256\}$, `line_words` $\in \{2, 4, 8, 16\}$, `ways` $\in \{1, 2, 4\}$) and every policy. The largest geometry holds $256 \times 16 = 4096$ words; eight times that is 2^{15} words, more than the R4 default `ADDR_W = 10` can address: run the sweep with `PARAMS="ADDR_W=16 DW=32"` (or limit the range to the address space and say so).

Transparency proves the model is *functionally* correct; it does not prove that hit/miss and memory traffic are right — a model that misses on every access is perfectly transparent. Step 1 covers that; so do two cheap invariants you must check after every access:

- `hits + misses == accesses`, and `writebacks <= misses` (before any flush);
- an access repeated immediately after any access to the same line is a `HIT` with no memory transactions.

4.3 Step 3: Clone Check

TEST TERM | Clone check

> A property test of the object structure, not of the cache: two copies must behave identically and independently. It catches aliasing (shared lines, a shared policy or generator, a shared memory) that no single-instance test can see.

Clone the model in the middle of a trace, continue both copies with the same remaining trace, and check that their predictions are identical (results, data, memory transactions, statistics). Then continue only the original and check that the clone did not change (compare `convert2string()` of the clone before and after). This catches shallow copies of lines, of the policy (including the random generator state) and of the backing memory. Lab T-AM1 relies on `clone()` to "warm up" a cache once and then run several experiments from the same warm state, and Lab T-DV1 uses it to checkpoint the model.

NOTE | Why three steps?

> Each step catches a class of error the others miss: hand traces catch a wrong *policy* or wrong *traffic* that is still functionally transparent; transparency catches data bugs that short traces never reach; the clone check catches aliasing bugs that appear only when two model instances exist — which is exactly the situation in T-DV1 (checkpoints) and T-AM1 (many configurations).

4.4 Bug-Injection Exercise for the Model

TEST TERM | Negative / error-injection test

> Here the design under test is the model itself: each injected bug must make at least one of the three steps fail. A bug that passes every step is a hole in the verification, not a harmless bug.

Inject each bug below into a copy of the model, one at a time, and record which step detects it. A bug that no step detects shows a hole in your traces; add a trace and re-run.

Injected model bug	Detected by step
Write-back uses the new tag instead of the victim's tag	
Refill data copied with `lines[s][w].data = src.data` from a shared line object	
LRU `touch()` not called on a hit	
FIFO treats a hit like a fill	
`clone()` copies the `lines` array of handles, not the lines	
`random_policy` uses `$urandom` instead of its own generator	
Write hit does not set the dirty bit	

5 Requirements and Tasks

1. Create `cache_model_pkg` with the classes of Section 3 in `dv/model/cache_model/`, one class per `.svh` file, and a file list `dv/model/cache_model/cache_model.f`.
2. Implement `cache_cfg` with `from_plusargs()`, `is_valid()` and `make_policy()`; an invalid configuration ends the simulation with `$fatal` and a clear message.
3. Implement `cache_line`, `mem_txn`, `cpu_txn`, `cache_pred`, `cache_stats` and `mem_model`, each with `convert2string()` and a deep `clone()` where it holds handles or arrays.
4. Implement the abstract `replacement_policy` and the three policies (requirements M3, M4). Show polymorphism: the model never mentions a derived policy class.

5. Implement `cache_model` with the API and algorithm of Section 3, satisfying M1–M8.
6. Write `dv/model/cache_model/tb/tb_model_unit.sv` with the tests `hand`, `transparency`, `clone` and `policy`, selected with `+TEST=`. Each prints its configuration and ends with exactly one `TEST PASSED` or `TEST FAILED` line, and has a watchdog (a maximum number of accesses, since there is no clock).
7. Write at least ten hand-calculated traces (Step 1) including the two examples above.
8. Run the transparency test for all 48 configurations and all three policies, at three seeds, with `make regress`, and record the summary (a regression, as in Lab V1: one command, a pass/fail table; one run may loop over all configurations, e.g. with a `+SWEEP=1` plusarg).
9. Run the bug-injection exercise and fill in the table.
10. Measure the model throughput (accesses per second of wall-clock time) for the default configuration; T-AM1 compares it with the RTL.
11. Optional: implement `plru_policy` (tree pseudo-LRU) without touching `cache_model.svh`, and show with a hand trace where it differs from true LRU with `ways = 4`.

6 Repository and Flow

The model is not an IP block, so it lives under `dv/model/` (owned by the architecture-modeling team, used by both tracks), but it follows the per-IP conventions of the repository: a README, a Makefile that includes the shared `flow/mk/` fragments, file lists rooted at `${REPO_ROOT}`, and the designer-level unit testbench in `tb/`.

```
dv/model/cache_model/
+-- README.md                  scope, API, class diagram, verification results
+-- Makefile                   IP := cache_model; includes common.mk, sim_verilator.mk,
    regress.mk
+-- cache_model.f              package file list (used by T-DV1, T-DV2, T-AM1, T-AM2)
+-- cache_model_pkg.sv         package: typedefs + `include of all class files
+-- cache_cfg.svh  cpu_txn.svh  mem_txn.svh  cache_line.svh  cache_stats.svh
+-- cache_pred.svh mem_model.svh replacement_policy.svh  cache_model.svh
+-- tb/
|   +-- tb_model_unit.sv       model-only testbench: hand, transparency, clone, policy
|   +-- hand_traces.svh        hand-calculated traces and expectations (data)
|   +-- tb.f
+-- regress.list               tests x seeds x PARAMS sets for `make regress`
```

```
// dv/model/cache_model/cache_model.f
+incdir+${REPO_ROOT}/dv/model/cache_model
${REPO_ROOT}/dv/model/cache_model/cache_model_pkg.sv

// dv/model/cache_model/tb/tb.f
-f ${REPO_ROOT}/dv/model/cache_model/cache_model.f
+incdir+${REPO_ROOT}/dv/model/cache_model/tb
${REPO_ROOT}/dv/model/cache_model/tb/tb_model_unit.sv
```

```
# dv/model/cache_model/Makefile
# (comments on their own lines: make keeps the spaces before a '#' in a value)
IP        := cache_model
TB_TOP    := tb_model_unit
# "lint" lints the package, through the testbench that imports it
RTL_FLIST := cache_model.f
TB_FLIST  := tb/tb.f
```

```
LINT_TOP   := tb_model_unit
LINT_FLIST := tb/tb.f
# compile-time widths only; the geometry is given with +plusargs
PARAMS    ?= ADDR_W=10 DW=32
TEST      ?= hand
REGRESS_LIST := regress.list
include $(REPO_ROOT)/flow/mk/common.mk
include $(REPO_ROOT)/flow/mk/lint.mk
include $(REPO_ROOT)/flow/mk/sim_verilator.mk
include $(REPO_ROOT)/flow/mk/regress.mk
```

No new make targets or variables are introduced; the existing ones apply:

```
source env.sh
make -C dv/model/cache_model lint
make -C dv/model/cache_model sim TEST=hand
make -C dv/model/cache_model sim TEST=transparency SEED=7 \
     PLUSARGS="+NUM_LINES=64 +LINE_WORDS=8 +WAYS=2 +POLICY=fifo +NUM_TXN=100000"
make -C dv/model/cache_model sim TEST=clone PLUSARGS="+WAYS=4 +POLICY=random"
make -C dv/model/cache_model regress          # summary in build/cache_model/
    regress_summary.txt
```

The four tests of `tb_model_unit` (`+TEST=`):

Test	What it shows
`hand`	Step 1: every hand trace replayed; every result, read word, way, memory transaction and statistic equal to the paper calculation (directed)
`transparency`	Step 2: random trace, model vs. plain memory, invariants after every access, memories equal after `flush()` (random self-check)
`clone`	Step 3: clone mid-trace; identical continuation; the clone unchanged while the original runs
`policy`	the policy interface alone: `ways = 1` gives way 0, short LRU/FIFO sequences, random policy reproducible from its seed, `make_policy()` and `$cast`, invalid configurations rejected

SYNOPSYS CLOUD

On the cloud `SIM=vcs` is the default and the only simulator that compiles the package (see the tool note in Section 2); the regression of the reference solution (48 geometries × 4 policies × 3 seeds plus the hand, clone and policy tests) runs in under two minutes. `make lint` uses the VCS compiler (`LINT_TOOL=vcs`, `+lint=all`), because SpyGlass lints synthesizable RTL only.

Output goes to `build/cache_model/<cfg>/sim/<test>_s<seed>.log` as for every IP. The top-level `make check-all` must include `dv/model/cache_model` from now on (`make lint sim TEST=hand`).

Git: branch `lab/tmod-cache-model`, merge request reviewed by the coordinator *and* one member of the architecture-modeling team, tag `tmod-done`. After the tag, changes to the model's API need a review by both tracks, because T-DV1 and T-AM1 depend on it.

7 Deliverables

The merge request contains the package, the unit testbench, the regression list and `dv/model/cache_model/README.md` with:

- A class diagram: inheritance (transactions, policies) and ownership (which object creates and holds which: model → lines, policy, memory, statistics).

- The model's scope: what it predicts and what it leaves out, and why (the table of Section 3, in your own words).
- The API with one usage example (ten lines of SystemVerilog that create a model, run three accesses and print the statistics).
- The word order of refill and write-back, and confirmation that it matches the R4 RTL.
- The three model verification steps and their results: the hand-trace list, the transparency regression summary (configurations × policies × seeds) and the clone check.
- The bug-injection table, and any trace you added because of it.
- The measured model throughput (accesses per second, machine and configuration stated).
- Known limitations (for example: no byte enables, no timing, no coherence — the same out-of-scope list as R4).

8 Assessment

Criterion	**Weight**
Cache model: correctness, API, run-time geometry, statistics match R4 definitions	25%
Replacement policies behind the abstract interface; reproducible random policy	15%
Object-oriented quality: deep copies, virtual methods, encapsulation, package	20%
Model verification: hand traces, transparency, clone check	25%
Bug-injection exercise and throughput measurement	5%
README: class diagram, scope, results; repository conventions	10%
Total	100%

The optional pseudo-LRU policy adds up to 5 bonus points.

9 Review Questions

1. What is the difference between `b = a`, `b = new a` and `b = a.clone()`? Which one does your model need, and where?
2. What happens if `convert2string()` is not declared `virtual` and you call it through a base-class handle?
3. A derived transaction class does not override `clone()`. What type of object does cloning it return, and how would you notice?
4. Why is `replacement_policy` abstract? How would you add a pseudo-LRU policy, and which files would change?
5. Why does the policy interface have both `touch()` and `fill()`? Which policy needs the distinction?
6. Why may the random policy not use `$urandom`? What would the clone check report if it did?
7. Why are bit widths class parameters but the cache geometry a run-time object? What would break if geometry were a parameter?

8. Why does the model not predict timing? Which checks or labs cover timing instead?
9. A model that misses on every access passes the transparency check. Which of your checks catches it?
10. Your model and your RTL will agree on every transaction in T-DV1. How do you know the model is right?

10 References

- IEEE Std 1800-2017: Clause 8 (classes: handles, inheritance, virtual methods, abstract classes, parameterized classes, `$cast`), Clause 26 (packages).
- Chris Spear and Greg Tumbush, *SystemVerilog for Verification*: chapters on OOP basics and advanced OOP (copying objects, virtual methods, abstract classes).
- Verification Academy: SystemVerilog OOP material (deep copy, polymorphism, parameterized classes).
- Harris and Harris, *Digital Design and Computer Architecture: RISC-V Edition*: memory systems chapter, for the replacement policies.
- Hennessy and Patterson, *Computer Architecture: A Quantitative Approach*: Appendix B (cache basics, replacement).
- Verilator manual: supported class and dynamic-array features of the installed version.